

Territory-Aware Phase 1: Validating the Winner-Take-All Balance Term at High N

On the bad seed that was immune to raising λ , the balance term drives the entrenched run from -34% to *zero* imbalanced cells — but the discrete-area trim removes the energy plateau, so no level triggers mesh refinement, and the coarse level is shown to be resolution-floor-limited

July 2026

Status — measured (partial: the finest level was interrupted; confirmation is solid through level 1)

This report **confirms** the Stage-6 hypothesis of docs/plans/PHASE1_TERRITORY_AWARE_IMPLEMENTATION_PLAN.md: on the *bad* seed and λ that produced two entrenched -34% runs under the original energy, enabling the winner-take-all balance term drives the partition to 0 imbalanced cells (worst $\approx 2\%$), holding through a mesh transition. It also records two diagnoses of *why the run was slow*: (i) the discrete-area trim’s periodic retargeting removes the energy plateau, so no level fires the mesh refinement trigger — every level runs to the iteration cap; and (ii) the coarsest level is resolution-floor-limited (worst deviation floors at $\approx 10\%$, above the 5% gate), so refinement is *necessary*, not merely faster. Numbers are independently recomputed from the run’s trace snapshots. **Partial:** the run died mid-level-2 (host interruption), so the formal finest-level completion gate was not reached; the confirmation rests on level 1’s completed cap and a level-2 mid-snapshot.

Addendum 2026-08-03 — Result 1 is confirmed only on *area*; the term causes a *connectivity* regression

The July conclusion “the balance term works” is measured and stands **for the quantity it measured** — discrete-area imbalance. It did not test *connectivity*. A later run shows the machinery drives area to target by handing cells **disconnected** territory: 14/200 fragmented cells, against 0 for the matched term-OFF control at the same seed, λ , and mesh. Together with a wall-clock cost that made a 5-level $N=200$ ladder unaffordable, this makes the term a *net regression* at $N=200$, where the original energy already produced a valid finalised partition. See §7. Conclusion 1 of §6 must be read with that qualification.

Provenance

Date	2026-07-20 (analysis of the run of 2026-07-17... 07-19)
Surface	torus $T(R=1.0, r=0.6)$, $N=200$, seed 84172851, $\lambda=9$, 3 levels, <code>init_method: seeded</code>
Branch	<code>feat/phase1-territory-aware-relaxation</code>
Source run	territory: <code>run.20260717_102306</code> , <code>config</code> <code>parameters/torus_200part_coarse_seeded_lam9_territory_test.yaml</code> (<code>wta_balance_enabled</code> $\gamma=7.0$, $p=2$; <code>wta_trim_enabled</code> period 200, damping 0.5, clamp 0.20; <code>pgd_reduced_gradient</code> 8 dual sweeps). Control: <code>run.20260712.224424</code> (same seed/ λ , original energy, no flags).
Producing script	<code>make_figures.py</code> (beside this report; trajectory arrays are the extracted measurement — see its header)
Method inputs	per-level trace snapshots <code>traces/*_level{0,1,2}_internal_data.hdf5</code> (density field x every 500 iters); the run's <code>relaxation.log</code>
Versions	Python 3.12.13, numpy 2.4.4, scipy 1.17.1, matplotlib 3.10.8, h5py 3.16.0
Anchor	control worst-cell deviation = -34.2% (2 imbalanced); territory level-1 end = 0 imbalanced, worst $\pm 2.1\%$ (recomputed, matching <code>detect_area_imbalance</code>)

1 Question

Phase 1 pins equal area on the *continuous* density field, but the deliverable is the *argmax* (winner-take-all) partition; at high N the discrete areas drift, producing “runt” cells that make the Phase-2 equal-area constraint infeasible (`docs/reference/winner_take_all_partition_gap.md`). The territory-aware term (`docs/math/07-phase1-wta-balance/`) adds a penalty on the discrete-area imbalance plus a periodic trim that retargets the projection toward exact discrete equality. **Does it fix an entrenched runt on a seed and λ known to be immune to raising λ — the “anti-lottery” property $N=1000$ needs?**

2 Method

For each saved snapshot of the density $x \in \mathbb{R}^{V \times N}$, we rebuild that level’s torus mesh (`TorusMeshProvider`), take the winner-take-all label $w_i = \arg \max_k x_{ik}$, assign each vertex its lumped mass v_i , and form the discrete cell area $A_k = \sum_{i: w_i=k} v_i$. The relative deviation is $(A_k - \bar{A})/\bar{A}$ with $\bar{A} = \sum_i v_i/N$; a cell is *imbalanced* if $|\text{dev}| > 5\%$. This is exactly the `detect_area_imbalance` computation. All numbers below are recomputed directly from the trace snapshots.

3 Result 1 — the mechanism works on the bad seed

The control (original energy, same seed/ λ) ends with worst-cell deviation -34.2% and 2 imbalanced cells. With the territory flags on, the balance improves monotonically across levels to **0 imbalanced cells** (worst $\pm 2.1\%$) by the end of level 1, and level 2 inherits and holds it (Table 1, Fig. 1). No cell dies.

4 Result 2 — why no level triggered mesh refinement

Both completed levels ran to the 30,000-iteration cap; the log contains no “Refinement triggered” line. The cause is the trim, not a stuck optimizer. Every `wta_trim_period = 200` accepted

Table 1: Winner-take-all discrete-area imbalance, recomputed per level.

Level (verts/cell)	snapshot	worst dev	$n_{\text{imb}} (>5\%)$
Level 0 (48)	iter 0	$-30.3\%/+54.0\%$	149
Level 0 (48)	iter 29 999 (cap)	$-10.1\%/+7.0\%$	4
Level 1 (124)	iter 0 (post-interp)	$-17.9\%/+14.5\%$	51
Level 1 (124)	iter 29 999 (cap)	$-2.1\%/+2.1\%$	0
Level 2 (237)	iter 0 (post-interp)	$-4.3\%/+4.6\%$	0
Level 2 (237)	iter 7 500 (died)	$-1.5\%/+2.4\%$	0
<i>Control</i>	final	-34.2%	2

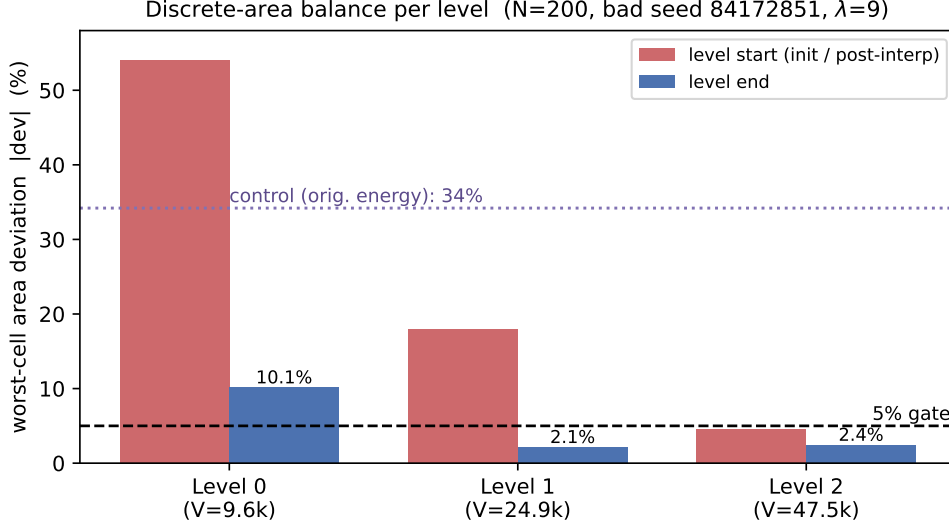


Figure 1: Worst-cell deviation at each level’s start and end vs the control. The mechanism clears the 5% gate; the control (dotted) stays at 34%.

iterations the trim retargets the per-cell area target d , re-projects, and re-baselines the energy — each retarget bumps E up before the next descent, so the energy **sawtooths and never plateaus** (level-0 tail: E oscillates $\approx 409\text{--}437$, amplitude ~ 8). The refinement trigger fires on an energy plateau ($\Delta E < 10^{-8}$ held for 30 checks); the sawtooth’s swings are nine orders of magnitude above that, so the plateau criterion is structurally impossible to satisfy. This behaviour is *new*: no standard/production config sets any of the three territory flags (verified), so previous $N=100/150/200$ runs plateaued cleanly and triggered normally. The extra iterations are not wasted spinning — the balance keeps improving — but the level cannot stop itself.

5 Result 3 — refinement is necessary, not merely faster

The trajectory of the worst deviation *within* each level (Fig. 2) settles the design question of whether one could simply run the coarsest level longer:

- **Level 0 is floor-limited.** Its worst deviation flatlines at $\approx 10\%$ — *twice* the 5% gate — with a tail rate of 0.23%/1000 iterations (pure sawtooth noise). The count of bad cells keeps falling ($14 \rightarrow 4$), but the *worst* cell is stuck, and the gate needs *every* cell under 5%. At 48 verts/cell the discretization cannot represent 200 equal areas below $\approx 10\%$; more iterations do not help.
- **Level 1 crosses the gate, steeply.** At 124 verts/cell the achievable floor drops to $\approx 2\%$; the first 5000 iterations fall at 2.2%/1000 ($\sim 10\times$ level 0’s tail), reaching 0 imbalanced by

~iter 9000.

The 5% gate lies *between* the two levels' resolution floors ($f_b \propto 1/\sqrt{V/N}$, the validity plan's band-fraction law), so refinement is what crosses it. Note the gate was crossable at just **124 verts/cell** here — the validity plan's mesh-budget floor guessed ~ 600 for margin, so ~ 250 – 300 finest may suffice (a 2–4 $\times$ compute rebate at $N=1000$, worth confirming).

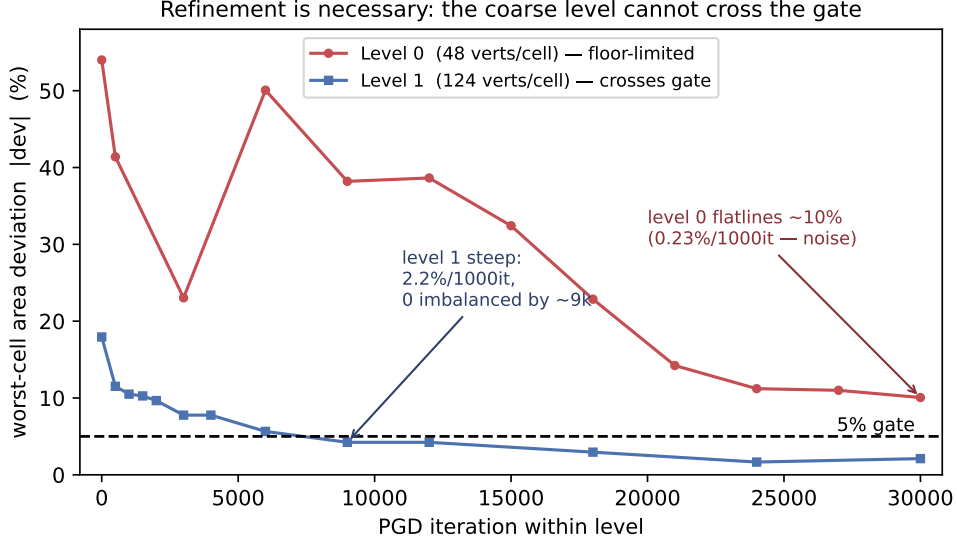


Figure 2: Worst-cell deviation vs iteration within levels 0 and 1. Level 0 floors above the gate; level 1 (finer) descends $\sim 10\times$ faster and crosses it.

6 Conclusions and implications

(*Conclusion 1 is qualified by the addendum of §7: it holds for area, not for connectivity.*)

1. **The balance term works** on the hardest case (bad seed, λ immune) — the anti-lottery property $N=1000$ requires. This confirms Stage 6.
2. **Its cost is diagnosed, not fundamental.** The slowness (level 0 alone 9.5 h, level 1 ~ 31 h, both to the cap) is the trim sawtooth preventing early triggering plus genuinely-continued optimization — both tunable.
3. **The coarse-only schedule needs the switch *after* level 1, gated on the imbalance metric, not fixed at level 0.** Level 0 cannot clear the gate (floor); the machinery must run until a level's floor drops below the gate (level 1 here), then the cheap original energy carries the already-balanced fine levels (level 2 started at 0 imbalanced). Dropping the trim on the cheap levels also restores clean plateaus and normal triggering. See docs/plans/PHASE1_COARSE_ONLY_WTA_SCHEDULE.md.
4. **Refine on a balance/stationarity plateau, not raw energy.** Level 0's balance flatlined by ~iter 22k while the trim-masked energy kept crawling; a gate-aware trigger would have refined there, saving ~ 3 h at no cost to the result.

7 Addendum (2026-08-03) — the connectivity regression and its cost

Provenance — addendum

Date	2026-08-03 (analysis of the run of 2026-07-30... 08-03)
Status	measured (mid-ladder: level 2 checkpoint; the run never reached its final level)
Surface	torus $T(R=1.0, r=0.6)$, $N=200$, seed 61803399, $\lambda=11$, 5 levels, <code>init_method: seeded</code>
Source run	territory: run_20260730_211516, config parameters/torus_200part_coarse_seeded_s61803399_lam11_adaptive.yaml (<code>wta_schedule: adaptive</code> , margin 0.03, re-arm 0.05), SLURM job 6371929 on UPPMAX Pelle, 4d wall. Control: run_20260722_175451 — <i>same</i> seed, <i>same</i> $\lambda=11$, <i>same</i> 5-level mesh, machinery OFF : 0 dead / 0 imbalanced / 0 fragmented, worst area deviation 1.7%, run to completion and finalised as a deliverable.
Producing script	testing/check_fragmentation.py (added for this analysis) on solution/checkpoint_level02.h5; relaxation.log for the timing and trim trace
Anchor	<code>n_fragmented = 14</code> with <code>n_imbalanced = 1</code> and <code>dead = 0</code> at level 2; worst cell 198, stray 43.57% of target

7.1 Why this addendum exists

The July report measured *discrete-area imbalance* and found the balance term fixes it. Connectivity was never tested. §4b of docs/reference/winner_take_all_partition_gap.md had by then recorded a third failure mode — **split cells**, whose winner-take-all territory holds the right *total* area in two or more disconnected islands — and attributed it to the seed lottery. This addendum tests that attribution, and finds it false.

7.2 Result 4 — the term fragments cells

The reseed run used the seed *proven clean* at this N , λ , and mesh. Checked mid-ladder at checkpoint_level02.h5 (levels 0–1 complete, $V=24,948$, `wta_active=True`):

	control term OFF, final	seed 84172851 term ON, final	seed 61803399 term ON, level 2
dead cells	0	0	0
imbalanced cells	0	0	1
fragmented cells	0	3	14
worst stray	—	57% (cell 58)	43.6% (cell 198)
most pieces	—	4	7 (cell 16)

These are not threshold artifacts. Target area is $4\pi^2 Rr/200 = 0.1184$; cell 198 holds $0.0663 + 0.0474 + 0.0042 = 0.1179$ — on target, in three places, its second piece 40% of a whole cell. Cell 198 does *not* appear in the imbalanced list. Only 15 cells have more than one raw component and 14 clear the 1% significance threshold, so the filter is barely binding.

The control is matched. run_20260722_175451 differs from the territory run *only* in the machinery flags. It finished all five levels with 0 fragmented cells and was finalised as a deliverable. The single remaining mismatch is the comparison level (2 vs. final), and it cuts against the term:

the control was clean *at the end*, while the territory run is already at 14 *mid-ladder*, with no mechanism that would heal it — nothing in the energy, the balance term, or the trim penalizes disconnection, so an area-balanced fragmented cell is a stable fixed point.

7.3 Result 5 — the causal trace points at the trim

`relaxation.log` records the trim’s worst-deviation target at each retarget. At level 0 it was dominated for hours by cell 198 (deviation $41.8 \rightarrow 44.4 \rightarrow 53.1 \rightarrow 55.8 \rightarrow 57.8 \rightarrow 61.2 \rightarrow 67.0 \rightarrow 63.7\%$) and cell 16 (48.2, 45.4, 38.3, 37.0, 35.4%), with cell 37 still flagged at the level end. Those are precisely the **two worst fragmented cells** at level 2 (198 at 43.6%; 16 in seven pieces), and 37 is in the flagged list. The trim drove their *total* area to target by giving them disconnected territory. It was also **clamp-saturated** — `d range` $[0.800, 1.200] \times \bar{A}$, exactly $\pm \text{wta_trim_clamp} = 0.20$ — for most of the run, i.e. pushing at maximum authority.

Consistent with this, the 3-fragment run `run.20260722_121925` saw its fragment count *fall* ($8 \rightarrow 6 \rightarrow 3 \rightarrow 3$) over exactly the levels that had handed off to the cheap original energy. In the present run the adaptive handoff **never fired** (level ends 12.69% and 8.31%, both above `wta_switch_margin` = 0.03), so the trim ran on *every* level.

7.4 Result 6 — the wall-clock cost

No level ever triggered mesh refinement; every completed level ran to the 30,000-iteration cap. The §6 conclusion 4 fix (refine on a balance/stationarity plateau) was implemented and *still* did not fire: $\|g_t\|$ flatlined at ≈ 20.33 , four orders above `refine_grad_tol` = 10^{-2} , and the plateau fallback needs 30 consecutive $|\Delta \|g_t|| < 10^{-4}$, which the trim’s every-200-iteration retarget breaks. Because the trim continuously moves the target, $\|g_t\|$ has no reason to approach zero — the balance-plateau trigger is structurally unreachable while the trim runs.

Level	mesh	V	iterations	wall time
1	100×96	9,600	30,000 (cap)	11.2 h
2	162×154	24,948	30,000 (cap)	53.5 h
3	224×212	47,488	9,500 (killed)	31.4 h
4	286×270	77,220	—	—
5	348×328	114,144	—	—

Four days of wall clock bought 2.4 of 5 levels. At the observed 11.9s/iteration, level 3 alone needs ≈ 99 h; extrapolating the measured scaling to levels 4–5 puts the full ladder at ≈ 20 days. The job was killed without ever writing a final solution or `metadata.yaml`, so none of the three validity gates ever ran — the 14 fragments were found only by checking the checkpoint directly.

7.5 Interpretation

At $N=200$ the original energy already produces a valid, finalised partition (the control). Against that baseline the territory-aware machinery is a **net regression on two axes**: it makes the run roughly an order of magnitude more expensive *and* it converts area-deficient cells into area-correct *disconnected* ones, which are strictly worse for Phase 2 (a split cell yields multi-loop contours the equal-area/Steiner machinery is not built for).

This does not retract Result 1. The term does what it was designed to do — it enforces *total* discrete area, and on the bad seed it did so where λ could not. The defect is that total area is the wrong invariant to enforce alone: area and connectivity must be enforced together, or fixing the first breaks the second. The motivating case for the term was $N=300$, which the original energy does not solve; the honest reading is that the term is **not yet ready** to be the answer there either, because it would carry this fragmentation with it.

No figure. The addendum's numbers come from a checkpoint that lives on the cluster and is not in this repository, so `make_figures.py` is unchanged and no new `fig_*.pdf` is generated. The measurement is reproducible with `testing/check_fragmentation.py` against that checkpoint.

8 Caveats

Single seed / single λ (by design — the point is the *bad* case). The run was interrupted mid-level-2, so the formal finest-level completion gate was not reached; a cluster re-run (with checkpoint/resume, and preferably the tuned schedule of conclusion 3) is needed to close it. The next A/B is N=300 vs `run_20260714.224821` (predicted $9 \rightarrow \leq 1$ over gate).

Addendum caveats (2026-08-03). The connectivity comparison of §7 is at level 2 for the territory run and at the final level for both controls; there is no term-OFF measurement at level 2 specifically. The decisive experiment is cheap and not yet run: relax seed 61803399, $\lambda=11$ to level 2 with `wta.schedule: off` and count fragments. Two seeds only. That the *trim* rather than the *balance term* is the fragmenting component is an inference from the log trace and the handoff correlation — the two have never been enabled separately, and `wta.balance_enabled` / `wta.trim_enabled` are independent flags, so that A/B is also available.